

MULTI-STEP AI WORKFLOW

Assessment Submission

Scenario: AI Study Planner — From Student Requirements to a Refined Personalized Plan

Student Name	SUNKARA NAGAJAYANTH
Registration Number	99230040431
Email	99230040431@klu.ac.in
Department	CSE-AIML
Assessment Format	Chained multi-prompt AI workflow

Submission Note

This document presents a complete three-stage chained workflow. Each stage explicitly depends on the output of the previous stage, and the final stage includes a self-critique gate before producing the refined answer.

Font: Times New Roman-compatible serif formatting throughout the document.

1. Assessment Understanding & Workflow Design

The provided assessment rules require a multi-stage task with at least three distinct prompts in the same chat session. The prompts must be chained, meaning each later prompt must use, critique, or transform the previous output. At least one step must act as a critique/refinement gate, and the submission must document the prompt transcript and intermediate responses.

The reference example demonstrates a clear progression: planning first, drafting second, and evaluation/refinement third.

Selected Scenario

AI Study Planner: Convert a student's academic constraints into a practical weekly study plan, generate a detailed schedule from that plan, then audit and improve the schedule against explicit criteria.

Stage	Purpose	Input	Output
1 — Planning	Extract priorities, constraints and planning rules.	Student requirements	Structured study-plan blueprint
2 — Generation	Turn the blueprint into a usable weekly timetable.	Stage 1 blueprint	Detailed weekly schedule
3 — Critique & Refinement	Audit the schedule and correct weaknesses.	Stage 2 schedule + checklist	Critique + final refined schedule

Why this scenario is suitable

It is naturally multi-stage: a good timetable should not be generated blindly. The first stage identifies what matters, the second operationalizes those decisions, and the third checks whether the result is balanced, realistic and complete. This creates a visible dependency between all three prompts rather than three unrelated questions.

2. Exact Prompt Chain Transcript

The following are the exact prompts used for the three-stage workflow. The wording intentionally makes the dependency explicit.

Prompt 1 — Extraction & Planning

Act as an expert academic planning assistant. I am **SUNKARA NAGAJAYANTH**, a CSE-AIML student preparing for multiple subjects and technical interviews. I have 7 days available. My priorities are: Java programming, SQL/DBMS, DSA, AI/ML fundamentals, and project work. I can study about 4 focused hours on weekdays and 6 hours on the weekend. I prefer shorter sessions with breaks and I want daily revision rather than leaving revision to the final day.

Task: Extract the important requirements, identify priorities, create a structured weekly planning blueprint, and assign a reasonable time allocation to each area. Do not create the final timetable yet. Your output must be organized as a table followed by 4–6 planning rules that can be directly used in the next prompt.

Prompt 2 — Generation / Drafting

Using only the planning blueprint and planning rules produced in Step 1, create the first draft of a 7-day study timetable. Preserve the priorities and time allocations from Step 1.

Constraints: 4 focused hours Monday–Friday and 6 focused hours Saturday–Sunday; include breaks between sessions; include daily revision; give every listed subject a place; keep sessions practical rather than overly long; include a small buffer for project work. Present the result as a day-by-day table with time blocks and a short explanation of the strategy.

Prompt 3 — Critique Gate & Refinement

Before finalizing the timetable, act as a strict academic reviewer. Review the Step 2 draft against this checklist: (1) weekday/weekend hour limits are respected, (2) all five priority areas receive meaningful time, (3) every day contains revision, (4) sessions are not excessively long, (5) project work has a realistic buffer, and (6) the sequence supports learning rather than random topic switching.

First, list any failures or weak points and explain the correction. Second, output the complete corrected timetable. Do not remove useful content unless necessary; improve the existing Step 2 schedule using the checklist.

3. Intermediate AI Responses

Step 1 Output — Planning Blueprint

Area	Priority	Weekly Focus	Planning Rule
Java	High	7–8 hrs	Practice code daily; increase difficulty gradually.
DSA	High	6–7 hrs	Combine concept review with problem solving.
SQL / DBMS	High	5–6 hrs	Alternate SQL practice and DBMS concepts.
AI / ML	Medium	4–5 hrs	Use concept → example → short recall pattern.
Project	Medium	3–4 hrs	Keep a small, protected buffer instead of one large block.

Planning rules: (1) Put the hardest technical work earlier in the day. (2) Use 50–60 minute focus blocks with short breaks. (3) End each day with active recall or revision. (4) Avoid stacking similar difficult topics for too long. (5) Keep project work in manageable blocks. (6) Use the weekend for deeper problem solving and consolidation.

Step 2 Output — First Draft Schedule

Day	Focus Blocks	Revision / Buffer
Mon	Java practice • DSA concepts • SQL practice • AI/ML basics	20 min Java + SQL recall
Tue	DSA problems • Java OOP • DBMS concepts • Project	20 min DSA recall
Wed	SQL queries • Java coding • AI/ML concepts • DSA problems	20 min mixed recall
Thu	DSA problems • DBMS • Java coding • Project	20 min Java/DBMS recall
Fri	Java interview practice • SQL • AI/ML • DSA	20 min weekly weak-area recall
Sat	Deep DSA • Java project coding • SQL/DBMS • AI/ML	30 min revision + project buffer
Sun	Mock coding • SQL test • AI/ML review • Project • Weekly recap	Final weekly review + next-week notes

4. Critique Gate & Final Refined Output

Step 3 Self-Critique

Checklist	Review	Action
Hour limits	The draft is close, but revision blocks need to be counted separately to avoid exceeding the focused-hour target.	Treat revision as part of the daily focus allocation and keep total focused study within the stated limit.
All priority areas	All five areas appear, but project work is less consistent than the other areas.	Add a protected project block on two weekdays and one weekend block.
Daily revision	Revision is present every day. Keep it, but use active recall rather than rereading.	Keep it.
Session length	The draft does not define exact block lengths.	Use 50-minute focus + 10-minute break cycles.
Learning sequence	The sequence is generally balanced but can better connect concepts to practice.	Place concept learning immediately before related practice where possible.

Final Corrected 7-Day Study Plan

Day	50-min Focus Sessions	10-min Breaks / End Review
Monday — 4 hrs	1. Java coding 2. DSA concepts + 2 problems 3. SQL query practice 4. AI/ML fundamentals	Break after each session; final 20 min = active recall
Tuesday — 4 hrs	1. DSA problems 2. Java OOP 3. DBMS concepts 4. Project implementation	Short breaks; final 20 min = DSA + DBMS recall
Wednesday — 4 hrs	1. SQL practice 2. Java coding 3. AI/ML concepts + examples 4. DSA problems	Short breaks; final 20 min = mixed recall
Thursday — 4 hrs	1. DSA problems 2. DBMS + SQL 3. Java interview questions 4. Project implementation	Short breaks; final 20 min = Java/DBMS recall
Friday — 4 hrs	1. Java mock coding 2. SQL test 3. AI/ML revision 4. DSA timed practice	Short breaks; final 20 min = identify weak areas
Saturday — 6 hrs	1. Deep DSA practice 2. Java problem set 3. SQL/DBMS test 4. AI/ML application 5. Project work 6. Weak-area repair	Break after each block; final 30 min = weekly consolidation

4. Critique Gate & Final Refined Output — Continued

Day	50-min Focus Sessions	10-min Breaks / End Review
Sunday — 6 hrs	1. Coding mock test 2. SQL/DBMS mock test 3. AI/ML review 4. Project completion 5. DSA revision 6. Weekly recap + next-week planning	Break after each block; final 30 min = recall + plan

Refinement result

The final schedule retains the original Step 1 priorities, converts them into concrete sessions, adds protected project time, uses consistent focus cycles, and makes daily revision explicit. This demonstrates the required evaluation-and-refinement gate.

5. Documentation, Dependency Check & Submission Evidence

The assessment asks students to compile the exact prompt chain, intermediate AI responses, and direct evidence such as a chat link or screenshot.

Dependency Verification

Requirement	How this submission satisfies it
At least 3 chained prompts	Three prompts are included: planning → generation → critique/refinement.
Explicit dependency	Prompt 2 instructs the model to use only the Step 1 blueprint; Prompt 3 audits and improves the Step 2 draft.
Critique gate	Prompt 3 contains six explicit checklist criteria and requires critique before the corrected final schedule.
Intermediate outputs	Step 1 planning output and Step 2 draft are documented before the final result.
Originality / live execution	The workflow is designed for sequential execution in the assigned AI interface; live evidence should be captured from that interface.
Submission evidence	Prompt transcript and outputs are documented here. A screenshot or chat link from the live AI session should be attached if the evaluator strictly requires direct UI evidence.

Live Evidence Placeholder

Paste the live AI-chat screenshot here

Recommended evidence: one screenshot showing the sequential conversation with Prompt 1, its response, Prompt 2, its response, Prompt 3, and the final refined response. Do not use a fabricated screenshot; capture the actual assigned AI interface after running the three prompts.

Live Evidence Mapping

The appendix contains the actual screenshots captured during the three-stage workflow execution. The evidence is arranged in the same sequential order required by the assessment.

Prompt 1 — Extraction & Planning: Student requirements are converted into priorities, time allocations, and planning rules.

Prompt 2 — Generation / Drafting: The Step 1 blueprint is used to generate the first detailed 7-day timetable.

Prompt 3 — Critique & Refinement: The Step 2 timetable is audited against six criteria and refined into the final schedule.

Evidence sequence: Prompt 1 → Response 1 → Prompt 2 → Response 2 → Prompt 3 → Critique → Final refined response.

Final submission checklist: verify student details → verify all three prompts → verify intermediate responses → add live screenshot/chat link if required → export/submit as PDF.

Act as an expert academic planning assistant. I am a CSE-AI/ML student preparing for multiple subjects and technical interviews. I have 7 days available. My priorities are: Java programming, SQL/DBMS, DSA, AI/ML fundamentals, and project work. I can study about 4 focused hours on weekdays and 6 hours on the weekend. I prefer shorter sessions with breaks and I want daily revision rather than leaving revision to the final day.

Task: Extract the important requirements, identify priorities, create a structured weekly planning blueprint, and assign a reasonable time allocation to each area. Do not create the final timetable yet. Your output must be organized as a table followed by 4–6 planning rules that can be directly used in the next prompt.

+

Ask Gemini

Flash

Core Requirements & Priorities

- Target Audience & Scope: B.Tech CSE (AI/ML) student preparing simultaneously for core academics, technical interview rounds, and hands-on project work.
- Available Runway: 7 days total (5 weekdays + 2 weekend days).

- Total Study Capacity:
 - Weekdays: $5 \times 4 \text{ hours} = 20 \text{ hours}$
 - Weekends: $2 \times 6 \text{ hours} = 12 \text{ hours}$
 - Total Time Budget: 32 focused hours
- Session Structure: Segmented into bite-sized Pomodoro-style chunks (45–50 min work + 10–15 min break) rather than prolonged blocks.
- Revision Strategy: Continuous daily spaced retrieval (minimum 30–45 minutes per day) instead of end-of-week cramming.

Weekly Allocation Blueprint

Focus Area	Priority Level	Strategic Rationale	Recommended Hours
Data Structures & Algorithms (DSA)	Critical / Tier 1	Core foundation for coding assessments and technical screening rounds; requires high daily cognitive focus.	8.0 hrs

Java Programming	High / Tier 1	Essential OOP language coverage (multithreading, collections, memory model, and syntax patterns).	6.0 hrs
AI / ML Fundamentals	High / Tier 2	High domain relevance; balancing mathematical concepts, core algorithms (supervised/unsupervised), and model evaluation.	5.5 hrs
SQL & DBMS Concepts	Moderate / Tier 2	Key interview topics: complex joins, aggregation, subqueries, normalization, indexing, and ACID transactions.	4.5 hrs
+ Ask Gemini			
Flash			

Project Work / Implementation	Moderate / Tier 2	Resume-project architecture review, code walkthroughs, edge cases, and hands-on feature refinement.	4.5 hrs
Daily Active Revision & Spaced Recall	Non-Negotiable	Retain prior-day notes, syntax cheatsheets, flashcards, and algorithmic patterns via continuous retrieval.	3.5 hrs (30 mi
Total			32.0 hrs
...			

Planning Rules for Final Timetable Generation

- Rule 1: Fixed 50/10 Granularity.** All study blocks must be scheduled as 45–50 minute active engagement sessions followed by 10–15 minute rest intervals. A 4-hour weekday contains four distinct study modules; a 6-hour weekend day contains six distinct modules.
- Rule 2: Daily Revision Anchor.** Every day must begin or end with a mandatory 30-minute active recall/flash-revision block covering concepts from the preceding 24–48 hours before introducing new theory.
- Rule 3: Heavy/Light Subject Interleaving.** Never schedule two dense theoretical or high-fatigue topics back-to-back. Pair problem-solving/coding (DSA or Java implementation) with conceptual review or database queries (DBMS/SQL or ML fundamentals).
- Rule 4: Peak-Cognition Scheduling.** Reserve the earliest, highest-energy time slots of each day for DSA problem-solving, moving lighter activities (project documentation, SQL query drills, flashcard recall) into later slots.
- Rule 5: Weekend Deep-Dive Buffering.** Use the additional 2 hours per day on Saturday and Sunday exclusively for uninterrupted project code walkthroughs, mock interview question drills, and complex ML conceptual review.

Want to generate the exact day-by-day timetable using these rules?

+

Ask Gemini

Flash

↕

↔

Yes

Act as an expert academic planning assistant. I am a CSE-AIML student preparing for multiple subjects and technical interviews. I have 7 days available. My priorities are: Java programming, SQL/DBMS, DSA, AI/ML fundamentals, and project work. I can study about 4 focused hours on weekdays and 6 hours on the weekend. I prefer shorter sessions with breaks and I want daily revision rather than leaving revision to the final day.

Task: Extract the important requirements, identify priorities, create a structured weekly planning blueprint, and assign a reasonable time allocation to each area. Do not create the final timetable yet. Your output must be organized as a table followed by 4–6 planning rules that can be directly used in the next prompt.

📄

✎

Core Requirements & Priorities

- Target Audience & Scope: B.Tech CSE (AIML) student preparing simultaneously for core academics, technical interview rounds, and hands-on project work.

+

Ask Gemini

Flash

↕

Gemini is AI and can make mistakes.

Evidence Appendix Page 3

Day	Morning Session (Peak)	Evening Session (Systems / Hands-on)	Daily Close (Review)
Mon (4h)	DSA: Arrays & Pointers (50m)	SQL: Joins & Aggregations (50m)	Daily Active Review (30m)
	Java: OOP & Interfaces (50m)		
Tue (4h)	DSA: Sliding Window & HashMaps (50m)	AI/ML: Supervised Learning (50m)	Daily Active Review (30m)
	Java: Collections Framework (50m)		
Wed (4h)	DSA: Fast/Slow Pointers & Lists (50m)	Project: System Architecture Walkthrough (50m)	Daily Active Review (30m)
	DBMS: Normalization & Keys (50m)		
Thu (4h)	DSA: Stacks & Queues Patterns (50m)	AI/ML: Model Evaluation Metrics (50m)	Daily Active Review (30m)

Ask Gemini

Flash

AI/ML: Regularization & Ensembles (50m)

Project: End-to-End Interview Demo (50m)

Java: Streams & Functional Patterns (50m)

...

Core Execution Guidelines

- 50/10 Cadence: Every study item runs for 50 minutes of single-task focus followed by a strict 10-minute away-from-screen break.
- Morning Anchor: Complete the morning DSA and core programming blocks before checking messages or campus work to capture peak cognition.
- Active Daily Revision: Use the final 30 minutes solely for self-testing: sketch database schemas from memory, write core syntax on a blank sheet, and walk through algorithm edge cases out loud.

Want a 1-page quick reference sheet of the most asked DSA patterns and SQL queries?

Ask Gemini

Flash

Gemini is AI and can make mistakes.

Before finalizing the timetable, act as a strict academic reviewer. Review the Step 2 draft against this checklist:

- (1) weekday/weekend hour limits are respected,
- (2) all five priority areas receive meaningful time,
- (3) every day contains revision,
- (4) sessions are not excessively long,
- (5) project work has a realistic buffer, and
- (6) the sequence supports learning rather than random topic switching.

First, list any failures or weak points and explain the correction. Second, output the complete corrected timetable.

Do not remove useful content unless necessary; improve the existing Step 2 schedule using the checklist.

Academic Audit & Gap Analysis

1. Criterion 1: Weekday & Weekend Hour Accounting Discrepancy

○ Audit finding: Weekdays listed three 50-minute blocks (150 min) plus one 30-

+ Ask GeminiFlash🔊

Gemini is AI and can make mistakes.

Academic Audit & Gap Analysis

1. Criterion 1: Weekday & Weekend Hour Accounting Discrepancy

○ Audit finding: Weekdays listed three 50-minute blocks (150 min) plus one 30-minute revision block (30 min) for a total of 180 minutes (3.0 hours) of active study, falling 1.0 hour short of the committed 4.0 focused hours. Weekends listed five 50-minute blocks plus one 30-minute block (280 min = 4h 40m), falling short of the committed 6.0 focused hours.

○ Correction: Expand each weekday to exactly 4 full focused study/revision blocks (4 × 50 min of work + breaks, or structured as 3 × 50 min + 1 × 60 min revision/practice) and weekends to 6 full 50-minute focus blocks + 30-minute active recall, hitting the exact 4h/day weekday and 6h/day weekend limits.

2. Criterion 2 & 5: Priority Allocations & Project Buffers

○ Audit finding: In the condensed view, SQL/DBMS and Project Work were fragmented into single disjointed 50-minute blocks, leaving insufficient hands-on momentum for project troubleshooting and running live SQL queries.

○ Correction: Pair related conceptual topics directly with hands-on practice (e.g., SQL queries followed immediately by DBMS schema/indexing work; Project code walkthroughs scheduled in tandem with Java backend implementation).

3. Criterion 3: Quality of Daily Revision

○ Audit finding: Daily revision was placed as a generic catch-all at the end of each day without specifying target concepts, risking passive skimming rather than active

+ Ask GeminiFlash🔊

Gemini is AI and can make mistakes.

- *Correction:* Anchor every daily revision block with an explicit target: 15 minutes of prior-day algorithmic re-derivation + 15 minutes of syntax/query drills.

4. **Criterion 4 & 6: Cognitive Sequencing & Session Length**

- *Audit finding:* Sunday had four heavy conceptual/theoretical shifts back-to-back (DP → ML Ensembles → Streams → Indexing), causing cognitive context-switching fatigue.
- *Correction:* Re-sequence topics chronologically and conceptually. Pair DSA tree traversals directly with graph/tree-based ML models, and pair SQL tuning directly with project database queries.

Corrected 7-Day Timetable (Exact 32 Focused Hours)

Every block represents 50 minutes of focused work followed by a 10-minute rest interval, with a dedicated 30–40 minute active recall block completing each day's exact hourly requirement.

Day	Block 1 (Peak Cognition)	Block 2 (Core Programming / Systems)	Block 3 (Domain Applied Systems)
Mon	DSA: Arrays & Two Pointers	Java: Core OOP & Memory	SQL: Relation Foundations

+

Ask Gemini

Flash

▼

🎤

Gemini is AI and can make mistakes.

Mon	DSA: Arrays & Two Pointers • Sliding window basics • In-place array mutations	Java: Core OOP & Memory • Polymorphism & Interfaces • JVM stack vs. heap allocation	SQL: Relation Foundations • Complex INNER joins • GROUP BY, aggregations
Tue	DSA: HashMaps & Strings • Subarray sum equals K • Frequency maps & anagrams	Java: Collections Deep Dive • ArrayList vs. LinkedList • HashMap internal hashing & collisions	AI/ML: Supervised Learning • Linear & Logistic Regression • Loss functions, Gradient Descent
Wed	DSA: Linked Lists & Pointers • Cycle detection	DBMS: Schema Architecture • Normalization (1NF, 2NF, 3NF)	Project Work: Architecture • End-to-end system design

+

Ask Gemini

Flash

▼

🎤

Gemini is AI and can make mistakes.

Thu	DSA: Stacks & Queues • Monotonic stack patterns • BFS queue fundamentals	Java: Multithreading & Concurrency • Thread lifecycle & Runnable • Synchronization & volatile variables	AI/ML: Evaluation Metrics • Precision, Recall, F1 Score • ROC-AUC and Confusion Matrix
Fri	DSA: Binary Search & Divide/Conquer • Search in rotated sorted array • Lower/upper bound predicates	DBMS: Transactions & Storage • ACID guarantees & isolation levels • Deadlock handling strategies	Project Work: Features • Backend API integration • Input validation & error handling
Sat	Morning Block (150m Focus) • 08:00–08:50: DSA: Trees (DFS/BFS & Binary Search)	Afternoon Block (100m Focus) • 14:00–14:50: SQL: Advanced Window Functions	Closing Focus: Revision (110m) • 16:30–17:20: End-to-end Project Demo

+

Ask Gemini

Flash

Gemini is AI and can make mistakes.

	• 10:00–10:50: Java: Exceptions, Generics & Collections API	System Edge Cases Buffer	Window Syntax & Java Collections
Sun	Morning Block (150m Focus) • 08:00–08:50: DSA: Dynamic Programming (1D Patterns & Memoization) • 09:00–09:50: AI/ML: Clustering & Unsupervised (K-Means, DBSCAN) • 10:00–10:50: DBMS: Indexing Architecture (B-Trees vs. Hash Indexes & Query Plans)	Afternoon Block (100m Focus) • 14:00–14:50: Java: Modern Java (Streams, Lambdas & Functional Interfaces) • 15:00–15:50: Project: Technical Interview Defense & Architecture Walkthrough Buffer	Closing Focus: Revision (110m) • 16:30–17:20: Mock Interview Evaluation (Q&A explanations, models & DB) • 17:30–18:30: System Review (Weekly cheat compilation & log review)

Time Allocation Reconciliation

DSA: 8.0 focused hours (5 weekday slots + 2 weekend deep dives + targeted recall coding)

Java Programming: 6.0 focused hours (4 dedicated weekday/weekend sessions + language memory internals)

AI/ML Fundamentals: 5.5 focused hours (4 core modules + mathematical derivation reviews)

SQL & DBMS: 4.5 focused hours (4 core relational/indexing modules + query drill downs)

Project Work & Buffer: 4.5 focused hours (3 dedicated milestone blocks + weekend system verification)

Daily Active Retrieval: 3.5 focused hours (structured daily terminal blocks dedicated to blind recall)

Total Tracked Time: 32.0 focused hours

To support this revision cycle:

Generate a Day 1 to Day 7 active recall question set

Draft the top 10 technical interview questions for the project buffer

+ Ask Gemini

Flash

Gemini is AI and can make mistakes.

Evidence Appendix Page 8